

Simple Web Survival Game — Codex Master Prompt

SUMMARY

Create a small, complete, playable 2D web survival game.

PROJECTS

SWARM SURVIVAL | Create an original top-down survival game playable in a browser using npm install and npm run dev. Target platforms include iPhone Safari, Android Chrome, and desktop browser.

- Player controls one character; enemies continuously move toward the player; player attacks automatically.
- Defeated enemies drop EXP; player collects EXP and levels up.
- Each level-up pauses the game and presents 3 upgrade choices.
- Survive for 5 minutes to win; HP reaching zero causes defeat.
- Use only 2 enemy types (Normal and Fast) with simple direct pursuit and no complex pathfinding.
- Implement EnemyManager with outside-screen spawning and gradual difficulty increase across timeline phases.
- Starting weapon is Pulse Gun with nearest-target auto-fire behavior.
- Use simple object pooling for enemies, player projectiles, and EXP pickups.
- Track Current Level, Current EXP, and EXP required for next level without losing extra EXP on level-up.
- Pause combat on level-up and present 3 randomly selected valid upgrades.
- Required collisions: player vs enemy, projectile vs enemy, player vs EXP; include short player damage cooldown.
- HUD includes HP (top-left), timer (top-center), level (top-right), and EXP progress bar.
- Implement mobile virtual joystick in lower-left with drag, normalized direction, visible base/thumb, and recenter behavior.
- Game timer is 5 minutes with victory at 00:00, enemy spawning stop, SURVIVED message, and Play Again button.
- Defeat screen shows GAME OVER plus Survival Time, Level, Enemies Killed, and Play Again.
- Main screen includes title SWARM SURVIVAL and PLAY button only.
- Use Phaser-generated graphics/shapes with no external artwork dependency.
- Include lightweight feedback effects (hit flash, death disappear, player damage flash, level-up overlay).
- Performance target: 60 FPS on modern smartphone with ~150–300 active enemies.
- Use compact specified file structure and centralize balance values in src/config/balance.ts.
- Implement development in phases 1 through 10, with testing and build/type checks after phases.
- Do not add out-of-scope systems such as backend, multiplayer, monetization, complex inventory, boss, shop, skins, etc.

TECHNICAL SKILLS

Phaser

TypeScript

Vite

HTML5 / WebGL

localStorage

CORE GAMEPLAY LOOP

Start Game ↓ Move Player ↓ Enemies Spawn ↓ Auto Attack ↓ Kill Enemies ↓ Collect EXP ↓ Level Up ↓ Choose 1 of 3 Upgrades ↓ Become Stronger ↓ Survive 5 Minutes ↓ Victory

FILE STRUCTURE

```
src/ |—— main.ts |—— scenes/ | |—— MenuScene.ts | |—— GameScene.ts |—— player/ | |—— Player.ts |——
enemies/ | |—— Enemy.ts | |—— EnemyManager.ts |—— weapons/ | |—— PulseGun.ts |—— systems/ | |——
ExperienceSystem.ts | |—— UpgradeSystem.ts | |—— ObjectPool.ts |—— ui/ | |—— HUD.ts | |——
VirtualJoystick.ts |—— config/ |—— balance.ts
```

TESTING REQUIREMENTS

1. Game loads. 2. PLAY starts game. 3. Player moves with keyboard. 4. Player moves using mobile joystick. 5. Enemies spawn. 6. Enemies move toward player. 7. Pulse Gun automatically fires. 8. Projectiles damage enemies. 9. Enemies die. 10. EXP drops. 11. EXP can be collected. 12. Player levels up. 13. Three upgrades appear. 14. Upgrade selection works. 15. Player can take damage. 16. Player can die. 17. Game Over works. 18. Timer reaches 00:00. 19. Victory works. 20. Play Again resets the game. 21. npm run build succeeds.

CODEX WORKFLOW

Inspect repository first. If empty, initialize project. If files exist, understand existing files before modifying. Implement only required phases. After each phase: run type checking, run application, fix errors caused by the phase. Do not rewrite unrelated working code.

FINAL OBJECTIVE

Open game URL on iPhone Safari; press PLAY; move using virtual joystick; fight automatically; kill enemies; collect EXP; choose upgrades; become stronger; survive 5 minutes; win or lose; press Play Again. Keep game small, understandable, and easy to expand later.